NIFTY 100

Analyst Guide

Candidate Analytics Pipeline, Financial Metrics, Clustering & API

Reviewer edition
Prepared as a practical guide to the project's data pipeline, analytical outputs, model methodology, application interfaces and validation evidence.

Project snapshot	Verified project evidence
Company universe	92 companies represented in the current analytical outputs
Sector coverage	10 broad sectors
Clustering	K-Means; 5 clusters; random_state = 42
API	16 GET endpoints
Automated tests	134 passed in the latest reported full test run
Performance test	16/16 endpoints returned PASS in a local sequential benchmark
Acceptance status	19/20 gates PASS; reviewer sign-off remains pending

Prepared: 30 October 2026
Primary database: **data/processed/nifty100.db**

Contents

01. Executive summary

02. Project scope and architecture

03. Data sources and data model

04. Data preparation and period handling

05. Financial KPIs and interpretation

06. CAGR and special-case rules

07. Clustering methodology

08. Generated reports and dashboard

09. FastAPI service reference

10. Testing and performance evidence

11. Limitations, risks and reviewer checklist

Appendix A. Key files and database tables

01. Executive summary

The Nifty 100 Candidate Analytics project is an analytical prototype that organizes company-level financial and market-related data into a structured database and exposes derived metrics through reports, a dashboard and a REST API. The project combines data preparation, financial screening, peer comparison, company-level visual summaries, portfolio-level statistics and unsupervised clustering.

The current project outputs cover 92 companies and 10 broad sectors. The clustering workflow produces five groups. The FastAPI application exposes 16 GET endpoints. The latest full automated test run reported 134 passing tests. A local sequential API benchmark reported all 16 endpoints passing, with zero recorded request errors.

The guide is intended to help a reviewer understand what the system does, where its outputs are stored, how to interpret the analysis and which limitations should be considered before relying on the results.

Important: This is a decision-support prototype, not an investment recommendation or a production-grade market-data service. Data lineage, accounting definitions and model stability must be independently reviewed before operational use.

02. Project scope and architecture

The project is organized as a pipeline: source workbooks and candidate analytical files are prepared, normalized and checked; the resulting structured data supports financial calculations and analytical outputs; those outputs are then made available through files, a dashboard and API routes.

Layer	Purpose	Typical project outputs
Source and ingestion	Read source Excel workbooks and analytical CSVs; preserve source lineage.	Source datasets; output/load_audit.csv
Normalization and validation	Standardize identifiers, periods and values; flag ambiguous or missing records.	output/validation_failures.csv; normalized data
Storage	Persist company, financial, sector and price-related data.	data/processed/nifty100.db
Analytics	Calculate financial ratios, peer comparisons, screens, portfolio summaries and clusters.	output/*.csv; output/*.xlsx; reports/*.png
Presentation	Present analytical results for review and exploration.	src/dashboard/app.py; PDF reports
Service/API	Expose selected analytical results through HTTP GET endpoints.	src/api/main.py; docs/openapi.json

High-level data flow

1. Inputs	→	2. Prepare	→	3. Analyze	→	4. Deliver
Excel / CSV source files	→	Normalize IDs, periods, values	→	Ratios, screens, peers, clusters	→	Database, dashboard, reports, API

The primary database path confirmed for this project is data/processed/nifty100.db. Earlier tracker references to data/nifty100.db should be corrected to this actual path.

Project scope includes analytical outputs for the current candidate universe. It does not, by itself, establish that every company is a current constituent of the official Nifty 100 index or that all input values are independently verified market data.

03. Data sources and data model

The analyst guide supplied with the project states that metrics are read from a period-aware candidate analytical CSV and cluster labels are read from output/cluster_labels.csv. The broader project also maintains source workbooks and a SQLite database. Source lineage should be retained so every reported metric can be traced to its input file, company identifier and reporting period.

Identifier standardization

• Trim leading and trailing whitespace from company_id.

• Convert company_id to uppercase before joins.

• Check join cardinality and unmatched identifiers after every major merge.

• Do not treat a successful file load as proof that records were inserted into SQLite; verify table counts separately.

Monetary convention

The project convention is to store monetary values in INR Crore. Reports should display this unit explicitly and avoid mixing rupees, lakhs and crores without conversion.

Simulated data disclosure

The project rules require stock_prices and market_cap simulated datasets to be clearly labelled SIMULATED in dashboards and reports. These fields should not be represented as verified live exchange data.

Database inventory

Table	Rows (last verified)	Purpose
annual_stock_returns	460	Annual return observations
company_scorecard	90	Company-level scorecard output
company_summary	92	Company summary records
derived_metrics	973	Derived analytical metrics
financial_panel	973	Period-aware financial panel
financial_ratios	1,076	Calculated financial ratio records
peer_percentiles	816	Peer-relative percentile outputs
sectors	92	Company sector classifications
stock_prices	5,520	Price observations; simulated-data disclosure applies
stock_returns	5,520	Return observations derived from price series

Counts are a snapshot from the project checks shared during development. They can change if the database is rebuilt or refreshed; re-run read-only database checks before a formal audit.

04. Data preparation and period handling

Data preparation is a critical control point because company-level financial records may contain inconsistent identifiers, missing values, ambiguous dates or multiple reporting periods.

Required preparation controls

• Read core Excel workbooks using the project convention **pd.read_excel(path, header=1)** where applicable.

• Normalize company identifiers before joins and retain a record of unmatched rows.

• Parse dates using explicit formats when the source format is known; do not rely on ambiguous automatic parsing.

• Preserve fiscal-year labels and distinguish annual, interim and trailing-twelve-month (TTM) periods.

• Investigate duplicates, missing values and implausible values before calculating ratios.

• Record the source file, source row or period, transformation and output field wherever practical.

Period handling

The supplied guide specifies that the clustering workflow accepts supported annual reporting-period labels. Interim and TTM periods must not be treated as annual observations. Fiscal-year alignment matters when comparing growth, profitability and balance-sheet values across companies.

Data-quality interpretation

A missing or ambiguous value is not automatically zero. A zero can have a real financial meaning, while a missing value indicates that the value was not available or could not be reliably interpreted. Review flags should therefore be investigated rather than silently converted.

Validation evidence

The project generated output/load_audit.csv and output/validation_failures.csv. The load audit previously recorded 12 source datasets successfully normalized, 12,892 source rows and zero load failures. This demonstrates successful source loading/normalization for that run; it should not be described as proof of database insertion unless the SQLite load is independently verified.

05. Financial KPIs and interpretation

Financial ratios are useful only when the numerator, denominator, reporting period and accounting definition are consistent. The exact project implementation should be checked against the calculation code and source fields before a reviewer treats a KPI as authoritative.

KPI	Common analytical meaning	Interpretation caution
Return on Equity (ROE)	Profitability relative to shareholders' equity.	Check whether average or closing equity is used and whether equity is positive.
Debt-to-Equity (D/E)	Balance-sheet leverage relative to equity.	Can be distorted by very small or negative equity; skip Financials sector in the D/E screener as specified by the project rules.
Operating Profit Margin (OPM)	Operating profit as a proportion of revenue/sales.	Confirm the operating-profit field and revenue basis are aligned.
Net Profit Margin	Net profit as a proportion of revenue/sales.	One-off items and tax effects may influence the result.
Asset Turnover	Revenue relative to the asset base.	Confirm average versus closing assets and period alignment.
Free Cash Flow	Cash generated after capital expenditure, according to the project's chosen definition.	Check cash-flow statement signs and capex treatment.
Interest coverage / debt-free status	Ability to meet interest expense from operating earnings.	When Interest Expense = 0, the project rule is to display "Debt Free" rather than divide by zero.

Financial-sector comparison

Banks and other financial companies have balance-sheet structures that differ materially from non-financial companies. For that reason, the project rule explicitly skips the Financials sector when applying the debt-to-equity screener filter. This is a screening convention, not a statement that leverage is irrelevant to financial institutions.

Interpretation discipline

• Compare companies within appropriate sectors and peer groups.

• Check whether values are annual and use the same fiscal-year basis.

• Treat extreme ratios as review flags, not automatic proof of misreporting.

• Do not rank companies solely on one ratio.

06. Five-year CAGR and special-case rules

Compound annual growth rate (CAGR) expresses the annualized rate of change between two observations separated by a defined number of years. The project's five-year CAGR logic requires observations exactly five years apart and valid endpoint values.

For a positive starting value and positive ending value:

CAGR = (Ending Value / Beginning Value) ^ (1 / 5) – 1

Project-specific handling

• If the base-year value is negative, return **TURNAROUND** instead of calculating CAGR.

• Do not calculate a five-year CAGR when the required endpoint observations are missing or not exactly five years apart.

• Apply the same unit and accounting definition to both endpoints.

• Do not interpret a high CAGR without checking the absolute starting and ending values.

A CAGR is not meaningful in the usual way when the starting value is zero or negative. The TURNAROUND label communicates a change in financial direction without presenting a mathematically misleading growth rate.

07. Clustering methodology

The project's candidate clustering workflow uses five selected features, StandardScaler and K-Means with five clusters and random_state=42. The supplied analyst guide identifies these settings. The exact feature names and preprocessing sequence should be confirmed from the clustering script and configuration used for the final run.

Step	Method	Why it matters
Feature selection	Five numerical features	Defines the dimensions along which companies are grouped.
Scaling	StandardScaler	Places features on comparable standardized scales before distance-based clustering.
Clustering	K-Means, 5 clusters	Partitions observations around cluster centroids.
Reproducibility	random_state = 42	Helps reproduce the same initialization behavior for the same data and environment.
Outputs	cluster_labels.csv and cluster_profiles.csv	Assigns companies to groups and summarizes group characteristics.

How to interpret the clusters

• A cluster is a group of companies with similar standardized feature patterns in the selected feature space.

• Cluster numbers are identifiers, not rankings. Cluster 0 is not inherently better than Cluster 4.

• Descriptive labels should be based on the observed cluster profiles, not assumptions about future returns.

• A cluster profile should be reviewed using original-scale values as well as standardized values.

• Small clusters are less stable and may represent unusual companies, outliers or data issues.

Known caveat

The earlier clustering output contained very small clusters, including clusters with only two companies and one company. These groups should be treated cautiously. The analyst should inspect the member companies and input values before assigning strong business interpretations.

Model validation questions for a reviewer

• Are the five input features economically meaningful and consistently available?

• Were missing values and outliers handled before scaling?

• Were the cluster profiles checked against original units?

• Was the choice of five clusters supported by the elbow plot or other diagnostics?

• Do cluster assignments remain broadly similar under reasonable changes to the input period or feature set?

Clustering is exploratory and descriptive. It does not establish causation, intrinsic company quality or expected investment performance.

08. Generated reports and dashboard

Output	Location	Purpose
Cluster labels	output/cluster_labels.csv	Company-to-cluster assignments.
Cluster profiles	output/cluster_profiles.csv	Summary statistics by cluster.
Elbow plot	reports/elbow_plot.png	Diagnostic to support selection of cluster count.
Correlation heatmap	reports/correlation_heatmap.png	Shows pairwise relationships among selected KPIs.

Output	Location	Purpose
Outlier report	output/outlier_report.csv	Flags unusual observations for investigation; not proof of error.
Portfolio statistics	output/portfolio_stats.csv	Descriptive summary of the current portfolio/universe.
Analyst guide	docs/analyst_guide.pdf	Explains methods, outputs and limitations.
Acceptance checklist	docs/acceptance_checklist.pdf	Records acceptance-gate evidence and outstanding approval.

Dashboard

The Streamlit application is located at **src/dashboard/app.py**. It is the interactive presentation layer for the project's analytical outputs. When demonstrating the dashboard, explain the source and period behind each displayed metric and identify simulated datasets wherever they are shown.

09. FastAPI service reference

The FastAPI application is located at **src/api/main.py**. The project exposes 16 GET routes for health checks, company data, screening, sectors, peers, clustering, outliers, valuation, portfolio statistics, data quality and report discovery.

Method	Endpoint	Purpose
GET	/health	Service health check.
GET	/companies	List company records.
GET	/companies/{company_id}	Retrieve one company's details.
GET	/companies/{company_id}/metrics	Retrieve company metrics.
GET	/screener	Apply screening criteria.
GET	/sectors	List sector information.
GET	/sectors/{sector_name}	Retrieve companies in a sector.
GET	/peers/{company_id}	Retrieve peer comparison data.
GET	/clusters	List cluster summaries.
GET	/clusters/{cluster_id}	Retrieve a cluster's details.
GET	/companies/{company_id}/cluster	Retrieve a company's cluster assignment.
GET	/outliers	Retrieve outlier flags/results.
GET	/valuation	Retrieve valuation-related outputs.
GET	/portfolio/stats	Retrieve portfolio statistics.
GET	/data-quality	Retrieve data-quality information.
GET	/reports	List report outputs.

API verification

The project's benchmark exercised all 16 endpoints using FastAPI TestClient. It used five warm-up requests and 50 measured requests per endpoint. All 16 endpoints returned PASS and the benchmark recorded zero non-200/error responses in the reported run.

The benchmark is a local, sequential, in-process TestClient measurement. It is useful as a regression and response-time baseline, but it is not a production network benchmark, concurrent load test or service-level guarantee.

OpenAPI specification

The exported API specification is stored at **docs/openapi.json**. Reviewers can use this file to inspect the route contract and response schemas. For a live demonstration, launch the service with the project's configured API command and open the interactive API documentation exposed by FastAPI.

10. Testing and performance evidence

Verification item	Latest reported result	Interpretation
Full pytest suite	134 passed; 2 warnings	No test failures in the reported run.
Sector regression tests	12 passed; 1 warning	Covers sector detail behavior and error cases.
API benchmark	16/16 PASS; 0 errors	All benchmarked endpoints returned successful responses.
Acceptance checklist	19/20 PASS	AC-20 reviewer sign-off remains pending.

Verification item	Latest reported result	Interpretation
Submission archive	432 files; ZIP integrity PASS	Archive passed its ZIP integrity test.

Warnings

• Starlette deprecation warning: current TestClient integration uses httpx in a way that Starlette has deprecated; the warning recommends installing httpx2.

• Pandas date-parsing warning: a date such as 31-03-2020 is parsed without explicitly setting dayfirst or a format. Use explicit date formats where possible.

These warnings did not cause failures in the reported test run. They should nevertheless be addressed in a maintenance pass to reduce future compatibility and parsing risk.

11. Limitations, risks and reviewer checklist

Key limitations

• Candidate-universe limitation: the current analytical output represents 92 companies; do not claim it is a complete 100-company universe without reconciling the source list.

• Data lineage: the guide describes candidate analytical CSV inputs and project-generated data. Verify original source provenance and update dates before external use.

• Simulated fields: stock_prices and market_cap must remain clearly labelled SIMULATED where applicable.

• Accounting consistency: ratio definitions can differ across data providers and accounting presentations.

• Period alignment: annual, interim and TTM values are not interchangeable.

• Clustering stability: very small clusters and outlier-sensitive features can make interpretation uncertain.

• Performance scope: the benchmark is local and sequential, not a concurrent production load test.

• Investment use: outputs are analytical indicators, not personalized investment advice or a guarantee of future performance.

Reviewer checklist

Review area	Questions to verify	Evidence to inspect
Data	Are source files, identifiers, units and periods traceable?	Load audit, validation failures, database tables.
Financial calculations	Are formula definitions and edge cases correctly implemented?	Ratio code, CAGR logic, output samples.
Screening and peers	Are filters, sector exclusions and peer definitions appropriate?	Screener config, peer comparison output.
Clustering	Are features, scaling, cluster count and profiles justified?	Clustering script, profiles, elbow plot.
Dashboard/API	Do displayed values match source outputs and route responses?	Dashboard demo, OpenAPI file, API tests.
Testing	Are tests reproducible and warnings understood?	pytest report, performance CSV and report.
Sign-off	Has the reviewer explicitly approved the deliverables?	Reviewer confirmation / sign-off record.

Acceptance status

The generated acceptance checklist currently records 19 of 20 gates as PASS. AC-20 — Reviewer sign-off — is intentionally PENDING until the assigned reviewer has inspected the work and formally approved it. Do not change this status based solely on successful tests or archive creation.

Appendix A. Key generated files

Artifact	Path
Primary SQLite database	data/processed/nifty100.db
Load audit	output/load_audit.csv
Validation failures	output/validation_failures.csv
Exploratory SQL	notebooks/exploratory_queries.sql
Financial ratios	financial_ratios table in primary SQLite database
Screener output/config	output/screener_output.xlsx; config/screener_config.yaml
Peer comparison	output/peer_comparison.xlsx

Artifact	Path
Radar charts	reports/radar_charts/
Dashboard	src/dashboard/app.py
Valuation and cash-flow outputs	output/valuation_summary.xlsx; output/cashflow_intelligence.xlsx
Pros/cons and parsed analysis	output/pros_cons_generated.csv; output/analysis_parsed.csv
Company tearsheets	reports/tearsheets/
Sector reports	reports/sector/ (10 reports verified)
Portfolio summary	reports/portfolio/
Cluster outputs	output/cluster_labels.csv; output/cluster_profiles.csv
API service/specification	src/api/main.py; docs/openapi.json
Test report	reports/pytest_report.html
Performance evidence	reports/performance/api_performance_results.csv; api_performance_report.md
Acceptance checklist	docs/acceptance_checklist.pdf
Submission archive	dist/nifty100_sprint6_submission.zip

Appendix B. Reproducibility and operating commands

Task	Command
Run test suite and HTML report	make test
Launch dashboard	make dashboard
Launch API	make api
Clean caches/test artifacts	make clean

Command note: use the Makefile only after checking its current implementation. During this project review, make load, make ratios and make report were found not to perform the actions their tracker descriptions imply. Prefer the verified Python scripts and explicit validation commands for those workflows.

End of Analyst Guide

Prepared for technical review. Verify current file versions, data lineage and reviewer approval before formal project sign-off.